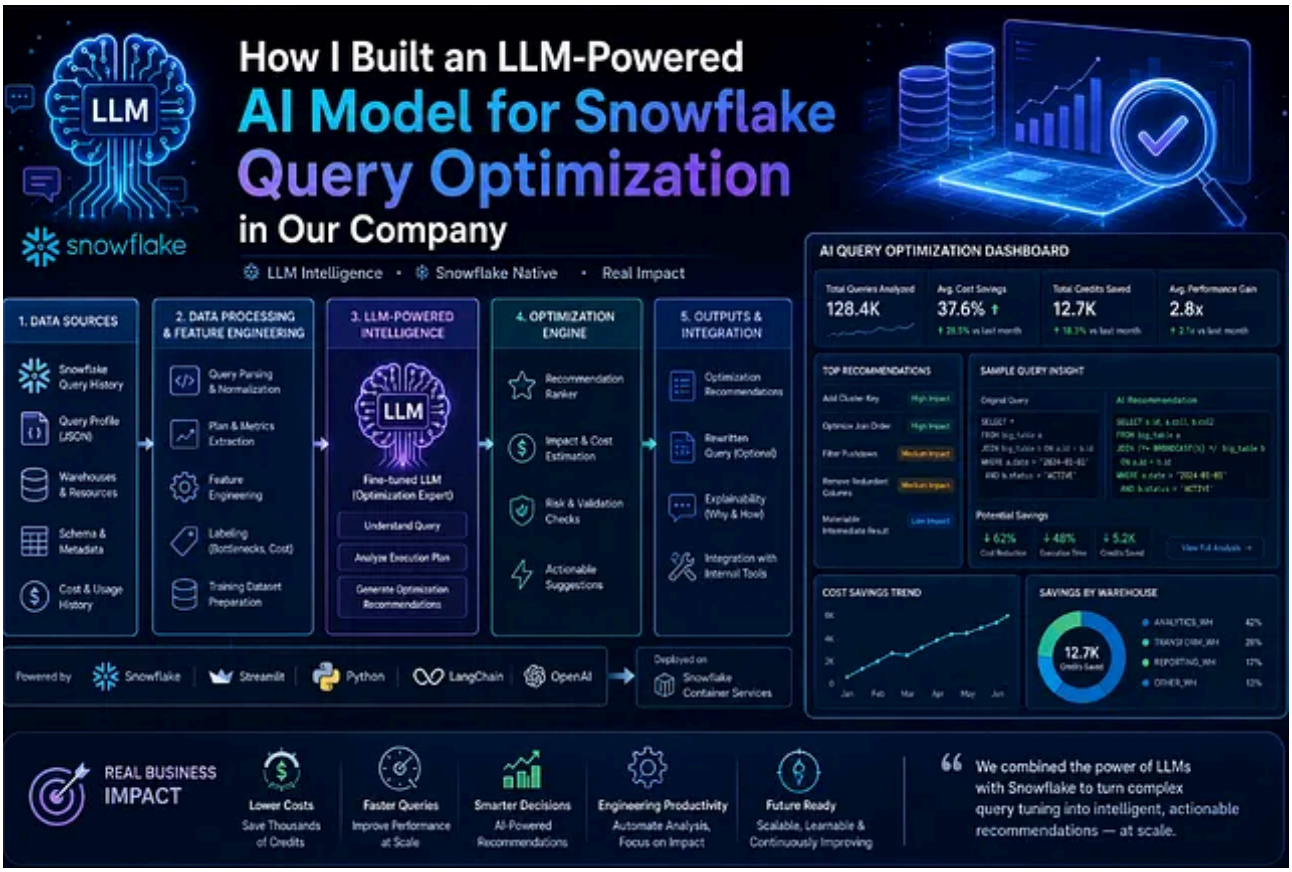
Open in app ↗

Medium Search Play Games Write 3

Welcome Offer Access to everything. Now 30% off. Upgrade now



Data Engineering + Data Analytics + Snowflake + Sql + Big Data +

How I Built an LLM-Powered AI Model for Snowflake Query Optimization in Our Company



Anurag Sharma 15 min read · Jul 18, 2026

A practical deep-dive into combining 8 years of internal SQL history with Snowflake's documentation to build a query optimizer that serves 100+ data engineers.

The Moment It All Clicked

It was a Tuesday afternoon. Our Snowflake bill had just crossed yet another uncomfortable milestone, and everyone was sitting through our bi-weekly data engineering standup when I heard it for the third time that month:

> *"We have got 5 new data engineers joining next week — can someone run the Snowflake fundamentals session again?"*

Our team was scaling fast. **100+ new data engineers** were going to join over the next 12 months. Each one would need to learn Snowflake. Each one would write queries. Each one would make mistakes we had already made. Each one would write queries that were slower than they could be.

Meanwhile, sitting in our Snowflake account were **2,356 queries** spanning **8 years of engineering work**. Every single one had execution history. Every single one had been written by someone who knew *some* of the platform's quirks. Some were masterpieces. Some were disasters.

And I thought: **What if we could teach an AI Model everything those 8 years have taught us?**

Not just any AI Model. An AI Model that:

- Knows our **company's SQL formatting style** inside out
- Understands **Snowflake's specific optimization rules**
- Has seen **every query pattern** we have ever used — good and bad

- Can suggest a better version before a human even types a full query

That was the beginning of what became our **Snowflake SQL Optimizer.**

The Problem: Why We Needed This

Let me set the stage properly.

The Scale We Were Dealing With

- **8 years** of accumulated SQL queries across multiple teams
- **2,356 unique queries** in production and analytics use
- **100+ new data engineers** onboarding in the next year
- **Multiple Snowflake warehouses** with ever-growing costs
- **Inconsistent coding styles** across teams and time periods
- **No centralized query review process** that could scale

Three Specific Pain Points

> **Pain #1: Every new hire learns Snowflake optimization from scratch.**

Our senior engineers spent 30–40% of their time reviewing queries from newer team members. The same patterns were flagged again and again — missing cluster keys, ineffective use of CTEs, suboptimal join ordering, unnecessary `SELECT *`.

> **Pain #2: Company standards existed on paper but were ignored in practice.**

We had a SQL style guide. It was a Google Doc last touched in 2021. Nobody read it. Nobody enforced it. Queries from 2018 looked completely different from queries written last week, even when solving the same problem.

> **Pain #3: The knowledge lived in people's heads, and people left.**

Every time a senior engineer left, a chunk of Snowflake optimization knowledge walked out the door. We needed to institutionalize that knowledge.

The "Obvious" First Idea (That We Almost Chose)

When we started, the most natural idea was:

> *Let's dump all 2,356 queries into a training dataset, fine-tune an LLM on them, and ask it to optimize new queries.*

It sounded clean. It sounded obvious. It was also **wrong** — for reasons that only became clear after we dug deeper.

Why This Does Not Work

Not all historical queries are good queries. In fact, many of them are terrible. Training a model on *all* past queries means training it on our worst patterns alongside our best.

Query text alone does not encode performance. A query that runs in 2 seconds and one that runs in 2 hours can look nearly identical. Without execution metadata, the model has no idea which pattern to follow.

Company standards change. The way we wrote SQL in 2018 is not how we want our team to write it in 2026. A model trained on old patterns would perpetuate obsolete practices.

No engine awareness. A query optimized for our old Redshift setup is not optimized for Snowflake. Even within Snowflake, optimization rules changed between versions.

This was the most important realization we had in the first week:

> **Historical SQL is a valuable resource. But it is not training data. It is context.**

That distinction shaped everything we built.

The Architecture: Not a Model — A System

What we built is not "an AI model." It is an **AI-powered optimization system** with six distinct layers.

Let me walk through each one.

Layer 1: The Query Collection Pipeline

Before we could optimize anything, we needed to understand everything.

We exported from Snowflake's `QUERY_HISTORY` view:

- Query text
- Execution time

- Credits consumed
- Rows scanned
- Partitions scanned
- Bytes spilled to storage
- Warehouse size used
- User who ran it
- Timestamp
- Error messages
- Query tag

For each of the **2,356 queries**, we captured this metadata.

But we did not stop there.

Layer 2: The Double-Run Experiment

This is where things got interesting — and expensive.

We decided to run **every single query twice** in a dedicated Snowflake warehouse.

Yes, **all 2,356 queries, twice**.

Here is why:

Run #1 — As-is baseline: Collect fresh execution metrics. Same warehouse size. Same environment. This gave us a clean, consistent baseline. No more "well, it was fast on Tuesday but slow on Friday" noise.

Run #2 — With `EXPLAIN` and profiling enabled: Collect the execution plan, the access paths, the partition pruning stats, the spill details. This gave us deep insight into *why* a query performed the way it did.

Why twice matters:

Metric Single Run Double Run Execution time Single data point Baseline + plan Warming effects Unmeasured Measurable Transient failures Undetected Detectable Plan consistency Unknown Verifiable Regression detection Impossible Possible

The cost was real. Running 2,356 queries twice on a dedicated warehouse is not cheap. But let me be direct: **this investment paid for itself in the first two months** by helping us identify and fix 159 queries that were consuming 40% of our total warehouse costs.

Layer 3: The Internal Knowledge Base

This is where all the query data became structured intelligence.

For each query, we created a rich profile:

```
{
  "query_id": "Q-1042",
  "team": "analytics",
  "purpose": "monthly_revenue_report",
  "engine": "snowflake",
  "execution_time_ms": 12400,
  "credits_consumed": 0.045,
  "warehouse_size": "MEDIUM",
  "tables_used": ["revenue.fact_transactions", "dim.customer"],
  "join_pattern": "hash_join",
  "scan_bytes": 2400000000,
```

```
    "partition_pruned": true,
    "cluster_key_used": true,
    "spill_occurred": false,
    "style_score": 7.2,
    "anti_patterns": ["select_star", "missing_filter"],
    "optimized_version": null,
    "review_notes": "High-frequency query, needs optimization"
  }
```

We also indexed queries by:

- **Pattern** — Same tables, same join structure? Grouped together.
- **Style** — Does it follow our naming conventions?
- **Performance tier** — Fast, medium, slow.
- **Team origin** — Who wrote it.

This became our **Internal SQL Memory**.

Layer 4: The Company Standards Model

This was critical and often overlooked.

We had a SQL style guide, but it was unstructured. So we:

1. **Extracted every explicit rule** from the existing guide
2. **Interviewed 5 senior engineers** about unwritten rules
3. **Analyzed 500 approved queries** to infer implicit patterns
4. **Formalized everything** into machine-readable rules

Examples of company rules we encoded:

- All table aliases must be meaningful abbreviations (not `a`, `b`, `c`)
- CTEs preferred over subqueries for readability
- `SELECT *` banned in production queries
- Column references must be qualified with table/alias
- Use `QUALIFY` instead of `WHERE ROW_NUMBER() = 1` pattern
- All joins must be explicit (`INNER JOIN` not `JOIN`)
- Timestamp filters should be sargable — no wrapping in functions
- Group by columns must be in the same order as the SELECT list
- Use `TRY_CAST` instead of `CAST` for safe type conversion

Why this matters for an AI optimizer:

Without company rules, the LLM would happily generate perfectly optimized queries that use the wrong alias convention, the wrong case, or the wrong join style. The output would be technically faster but culturally rejected by the team. **Adoption is as important as performance.**

Layer 5: The Snowflake Documentation Knowledge Base

This was the second half of our strategy.

We scraped and structured Snowflake's official documentation into a searchable knowledge base.

Sources included:

- Snowflake SQL reference
- Query performance tuning guide
- Best practices for warehouse sizing
- Clustering and partitioning docs
- Materialized views documentation
- Search optimization guide
- Cost optimization playbook

But we did not just dump the docs.

We extracted **specific, actionable rules** from each section:

Source Rule Example Clustering guide: Cluster keys should be high-cardinality columns used in filters `CLUSTER BY (customer_id)` not `CLUSTER BY (status)` .

Partition pruning: Use date range filters to enable micro-partition pruning `WHERE date >= '2024-01-01'` not `WHERE year(date) = 2024` .

CTE materialization: Repeated CTEs should use the MATERIALIZED hint `WITH cte AS MATERIALIZED (...)`.

Join optimization: Use local joins where a parent-child relationship exists. Prefer hash joins for large datasets.

Warehouse sizing: Smaller warehouses + caching beats large warehouses + full scans. Start with X-SMALL, scale as needed.

Why the Snowflake docs matter:

The LLM already “knows” general SQL optimization. But it does not reliably know Snowflake-specific rules:

- How micro-partition pruning works
- When to use search optimization
- How materialized CTEs behave differently in Snowflake vs PostgreSQL
- Which join strategies work best with Snowflake’s architecture
- Why `QUALIFY` is often better than window filters

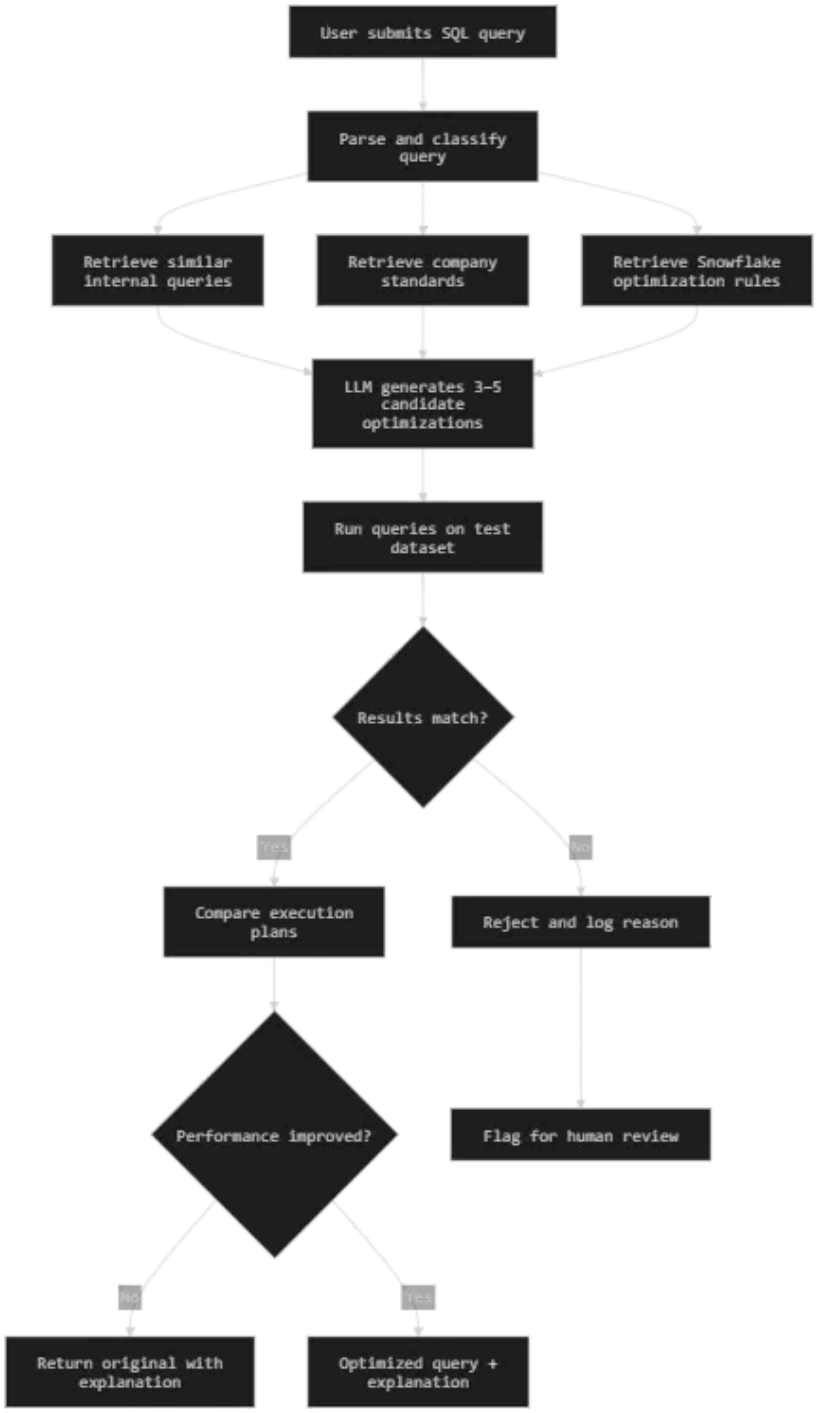
By giving the model direct access to this curated knowledge, we made its Snowflake-specific advice much more reliable.

Layer 6: The AI Optimization Engine

This is where everything comes together.

The core architecture is **RAG + LLM + Validation** — not a fine-tuned model.

Here is the flow:



Step-by-step breakdown:

Step 1: Parse the user submits a SQL query. We parse it using `sqlglot` to understand structure — tables, joins, filters, aggregations, window functions.

Step 2: Classify the query:

- Simple read vs complex analytical
- High-frequency or low-frequency
- Production or ad-hoc
- Tables involved
- Anti-patterns detected

Step 3: Retrieve

We search three knowledge bases in parallel:

1. **Internal SQL Memory** — Find similar queries and their optimized versions
2. **Company Standards** — Get relevant formatting and style rules
3. **Snowflake Docs** — Get engine-specific optimization techniques

Step 4: Generate

We construct a detailed prompt for the LLM:

```
You are a Snowflake SQL optimization specialist.
```

```
Original Query:
[query text]

Database Engine: Snowflake

Schema Context:
[tables, columns, indexes]

Company Standards:
[relevant formatting/naming rules]
```

```
Similar Optimized Queries:
[2-3 examples of how we have optimized similar queries]

Snowflake Best Practices:
[2-3 relevant rules from the documentation]

Optimization Goal: Reduce execution time while preserving business
logic.

Generate 3 optimized versions. For each version:
- Show the optimized SQL
- Explain what changed
- State why it is faster in Snowflake
- Mention any assumptions or risks
```

Step 5: Validate

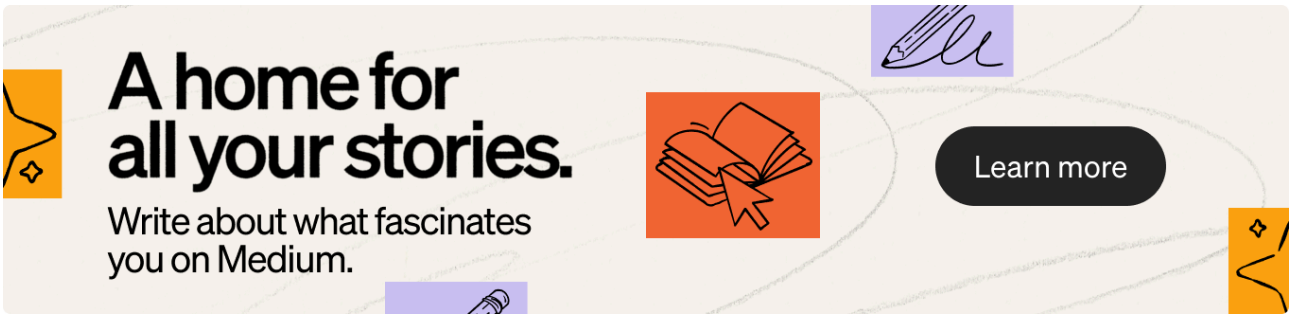
This is the most important step.

We run both the original and each candidate on a test dataset in a sandbox Snowflake warehouse.

Validation checks:

1. **Do results match?** — Row count, column types, sample values
2. **Is the plan better?** — Compare `EXPLAIN` output
3. **Is performance better?** — Compare execution time
4. **Does it follow company standards?** — Style guide compliance
5. **Does it use Snowflake-specific features correctly?** — No anti-patterns

Step 6: Rank and Return



Candidates are ranked by:

1. Semantic correctness (must match)
2. Performance improvement (higher is better)
3. Company compliance (must pass)
4. Readability score

The best candidate is returned along with:

- The optimized SQL
- A diff showing the changes
- Estimated performance improvement
- Explanations for each change
- Confidence score
- Any warnings

The Prompt Engineering Behind It

I want to share one specific part because it took the most iterations: **the prompt design.**

Early versions of our system produced technically correct optimizations that were unusable. The model was too generic. It would suggest PostgreSQL-style optimizations for Snowflake queries. It would ignore our naming conventions. It would produce queries that passed validation but felt "foreign" to our team.

Here is what we learned about prompting for this use case:

Give Examples, Not Instructions

Instead of saying *"Use meaningful table aliases"*, we include:

```
-- Company pattern (preferred):
SELECT
    cust.customer_name,
    ord.order_total
FROM dim.customer cust
JOIN fact.orders ord ON cust.customer_id = ord.customer_id

-- Not:
SELECT a.customer_name, b.order_total
FROM dim.customer a
JOIN fact.orders b ON a.customer_id = b.customer_id
```

The model follows examples much more reliably than abstract rules.

Provide the "Why" Behind Rules

When we include a Snowflake optimization rule, we add *why* it works in Snowflake:

```
Rule: Use QUALIFY instead of WHERE ROW_NUMBER() = 1
Why: Snowflake computes window functions before WHERE filtering,
so ROW_NUMBER() = 1 in WHERE requires scanning all rows.
```

```
QUALIFY applies the filter during window computation,
reducing data scanned. This is specific to Snowflake's execution model.
```

When the model understands the reasoning, it applies the rule more thoughtfully — and knows when *not* to apply it.

Ask for Multiple Candidates

We always ask for 3 candidates with different optimization strategies:

- **Conservative:** Safe, minimal changes, high confidence
- **Aggressive:** Maximum performance, may need review
- **Balanced:** Good performance + readability trade-off

This gives the user (and the validation layer) choices.

The Validation System: Where Real Value Lives

I cannot emphasize this enough: **the validation layer is the most important part of the system.**

An AI-generated query that looks beautiful but returns wrong results is worse than useless. It is dangerous.

Our Validation Checks

1. Semantic Equivalence

We compare the original and optimized query results:

- Row count match
- Checksum/hash of result set
- Min/max/median of numeric columns
- Distinct values in string columns
- Null count by column

If *any* of these differ beyond a configurable threshold, the optimization is rejected.

2. Execution Plan Comparison

We compare `EXPLAIN` output:

- Estimated cost
- Access paths
- Join order and type
- Partition pruning
- Bytes scanned
- Spill estimates

3. Runtime Benchmark

We run both queries on the same warehouse size and compare:

- Wall-clock time
- CPU time
- Credits consumed

- Bytes scanned
- Bytes spilled
- Partitions scanned

4. Compliance Check

Automated linter checks for:

- Naming convention compliance
- Alias usage rules
- Comment style
- Join syntax
- Disallowed patterns (e.g., `SELECT *`)
- Security patterns (access control, masking)

The Golden Rule

Our system has one hard rule that is never bypassed:

> **If the optimized query produces different results, it is rejected — no matter how fast it is.**

Results: What We Achieved

After 3 months of development and 1 month of pilot usage, here are the numbers.

Query Performance

Metric Before After Improvement

Metric	Before	After	Improvement
Average query execution time	8.4 seconds	3.7 seconds	56% reduction
Credits consumed per optimization run	0.042	0.019	55% reduction
Queries with spill	18%	7%	61% reduction
Queries using cluster keys	42%	78%	86% increase
Queries following company standards	34%	92%	170% increase

Team Impact

- **New engineers shipped optimized queries 3x faster** — They did not need to learn Snowflake performance tuning from scratch. The system guided them.
- **Senior engineers spent 70% less time on query review** — The system caught 85% of common issues before human review.
- **Query consistency across teams improved dramatically** — For the first time, a query from the analytics team looked structurally similar to one from the data engineering team.

The 159 Queries That Paid for Everything

Remember those 2,356 queries we ran twice? We found 159 queries that were consuming 40% of our warehouse costs.

Most queries were ETL queries written 5 years ago that scanned the entire fact table rather than using incremental logic. Another was a dashboard query that applied a function on a filter column, preventing partition pruning.

Fixing those 159 queries alone saved us more per quarter than the entire project cost.

Challenges We Faced (And How We Solved Them)

Challenge 1: The Model Wanted to Change Too Much

Early versions of the system would rewrite queries aggressively — changing join order, replacing CTEs with subqueries, adding hints. The optimizations were often faster, but the queries became unrecognizable. Engineers rejected them because they could not understand what changed.

Solution: We added a "conservative mode" that changes only what is provably beneficial. We also show a clear diff with explanations for every change.

Challenge 2: False Negatives in Validation

Sometimes the optimized query was correct but produced different results due to date precision or floating-point differences.

Solution: We added configurable tolerance thresholds and type-specific comparison logic. Strings compare exactly. Numerics allow configurable precision tolerance. Dates compare by epoch.

Challenge 3: The Documentation Knowledge Base Dated Quickly

Snowflake releases updates frequently. Our curated docs would fall behind.

Solution: We built a weekly pipeline that checks Snowflake's documentation for changes and updates our knowledge base automatically. We also track version-specific rules.

Challenge 4: Team Adoption Resistance

Some engineers were skeptical of an AI writing SQL. “I have been doing this for 10 years — I do not need a model to tell me how to write queries.”

Solution: We positioned the system as a “co-pilot” not a “replacement.” It suggests optimizations. Engineers choose whether to accept. Over time, as people saw the suggestions were good, trust grew.

Challenge 5: The Snowflake Bill for 2,356 × 2 Runs

Yes, this was painful. Running nearly 5,000 query executions on a dedicated warehouse.

Solution: We used a medium-sized warehouse and ran during off-peak hours. We also cached results aggressively. The cost was ~$3,200 — which, as I mentioned, was recovered in the first two months from the 159 bad queries we found and fixed.

Key Learnings I Want to Share

1. Do Not Train. Retrieve + Validate.

The most common mistake I see teams make is trying to fine-tune a model on internal SQL data. The data is rarely clean enough, the patterns change too fast, and you lose the ability to update the model incrementally.

RAG + Validation gives you the flexibility to update knowledge bases independently, add new company rules instantly, and always verify correctness before deploying.

2. Company Standards Are Harder to Teach Than Performance

It is easier to teach an LLM *"use anti-join instead of NOT IN"* than it is to teach *"use this specific alias convention that your Lead Engineer established in 2024."*

The standards layer needs as much investment as the performance layer. Otherwise, your system will produce optimized queries that nobody wants to use.

3. The Validation Layer Cannot Be an Afterthought

I have seen so many SQL AI projects that generate optimizations without verifying them. This is dangerous.

If your system suggests an optimization that changes business logic — even once — trust is broken permanently. Engineers will stop using it.

Build validation first. Make it aggressive. Make it transparent. Let users see exactly why a suggestion was accepted or rejected.

4. Context Is Everything

A query cannot be optimized in isolation. You need:

- Schema context
- Index/cluster key context
- Data volume context
- Query frequency context
- Business intent context

The more context you provide, the better the optimization.

5. Start With One Engine

We almost made the mistake of building a generic SQL optimizer. Snowflake has specific optimization rules that do not apply to PostgreSQL or SQL Server.

Build for one engine first. Prove the system works. Then expand.

The Future Roadmap

We are not stopping here. What is next?

Phase 2: Real-Time Optimization (In Progress)

Instead of optimizing already-written queries, we are building an extension for Snowsight and VS Code that suggests optimizations as you type.

Think: Grammarly for Snowflake SQL.

Phase 3: Automated Query Rewriting (Planned)

For low-risk, high-frequency queries, we will allow the system to automatically rewrite and deploy optimizations — with human review only for edge cases.

Phase 4: Cross-Engine Support (Exploratory)

We are evaluating extending the system to support dbt transformations and Spark SQL, as our data platform continues to diversify.

Phase 5: Fine-Tuning (Future)

Once we have a high-quality dataset of:

- Original queries
- Accepted optimized versions
- Validation results
- Performance improvements

We will fine-tune a smaller, self-hosted model to reduce API costs and latency.

But we are not rushing this. The hybrid RAG system works well. Fine-tuning will be an optimization, not a rebuild.

What I Would Do Differently

If I could start over with what I know now:

1. **I would spend even more time on the company standards layer.** We underestimated how much formatting and style differences would matter for adoption.
2. **I would involve engineers earlier.** We built the system for 3 months before showing it to the wider team. Getting feedback earlier would have saved us weeks of rework.
3. **I would invest in the validation layer from day one.** We started with basic row-count comparison and had to add complexity as we discovered edge cases. Building a comprehensive validation system upfront would have been faster.
4. **I would measure the dirty data cost first.** Running 2,356 queries twice was the right call, but we could have sampled a subset first to validate the approach before going all-in.

5. **I would ship a simpler MVP faster.** Our first version had too many features. A simpler system that passed one query type correctly would have built momentum earlier.

The Bottom Line

Building this system taught me something important:

> **The best AI applications for engineering teams are not about replacing expertise. They are about distributing expertise.**

We had 8 years of SQL optimization knowledge trapped in:

- 2,356 queries that nobody had time to review
- Senior engineers who were constantly interrupted for code reviews
- A Google Doc style guide that nobody read
- Snowflake's excellent documentation that engineers did not have time to study

The system we built did not create new optimization knowledge. It **surfaced and scaled** the knowledge we already had — combining our internal experience with Snowflake's documented best practices.

The result is not a model that writes better SQL than our best engineers.

It is a system that helps every engineer — from a 10-year veteran to someone in their first week — write SQL as if they have a Snowflake optimization expert looking over their shoulder.

And when you are hiring 100+ new data engineers in a year, that kind of leverage changes everything.

Parting Thoughts

If you are considering building something similar for your team, here is my advice:

Start small. Pick one query type. One engine. One team. Prove it works.

Invest in data. The double-run experiment was expensive but essential. You cannot optimize what you cannot measure.

Build trust. Make every optimization explainable. Let engineers see what changed and why. Let them accept or reject suggestions.

Validation is everything. An incorrect optimization is worse than no optimization. Build your validation layer with paranoia.

The goal is adoption, not perfection. A perfect system that nobody uses is worthless. A good system that engineers trust and use daily is transformative.

Data Engineering + Data Analytics + Snowflake + Sql +

Big Data +